`distrobox-create junit-maven -i fedora:latest`
got java and maven.

Standalone jar

in test_example directory, got the junit test jar for standalone testing as well as `Calculator.java` and `CalculatorTest.java`

compile

`javac -cp $CLASSPATH:./junit-platform-console-standalone-6.0.3.jar CalculatorTest.java Calculator.java

run test suite
`java -jar junit-platform-console-standalone-6.0.3.jar execute -cp .` --select-class MyTest`

```
📦[b@junit-maven Test_Example]$ java -jar junit-platform-console-standalone-
6.0.3.jar execute -cp . --select-class MyTest

💚 Thanks for using JUnit! Support its development at
https://junit.org/sponsoring


|
├─ JUnit Platform Suite ✘ TestEngine with ID 'junit-platform-suite'
encountered a critical issue during test discovery:
|
|        (1) [ERROR] ClassSelector [className = 'MyTest', classLoader =
null] resolution failed
|            Source: ClassSource [className = 'MyTest', filePosition = null]
|                    at MyTest.<no-method>(SourceFile:0)
|            Cause:
org.junit.platform.commons.PreconditionViolationException: Could not load
class with name: MyTest
|             at
org.junit.platform.engine.discovery.ClassSelector.lambda$getJavaClass$0(Clas
sSelector.java:100)
|             at
org.junit.platform.commons.function.Try$Failure.getOrThrow(Try.java:379)
|             at
org.junit.platform.commons.function.Try.getNonNullOrThrow(Try.java:199)
etc etc etc
```

No wonder that's failing. The test file is called "CalculatorTest" and the command we were given is trying to run a file called MyTest.

Proper commands:

```
📦[b@junit-maven Test_Example]$ javac -cp junit-platform-console-standalone-6.0.3.jar CalculatorTest.java Calculator.java

📦[b@junit-maven Test_Example]$ java -jar junit-platform-console-standalone-6.0.3.jar execute -cp . --select-class CalculatorTest
```

Output:

```
╷
├─ JUnit Platform Suite ✔
├─ JUnit Jupiter ✔
│  └─ CalculatorTest ✔
│     ├─ testDivideByZero() ✔
│     ├─ divide() ✔
│     ├─ add() ✔
│     └─ multiply() ✔
└─ JUnit Vintage ✔

Test run finished after 59 ms
[         4 containers found      ]
[         0 containers skipped    ]
[         4 containers started    ]
[         0 containers aborted    ]
[         4 containers successful ]
[         0 containers failed     ]
[         4 tests found           ]
[         0 tests skipped         ]
[         4 tests started         ]
[         0 tests aborted         ]
[         4 tests successful      ]
[         0 tests failed          ]
```

Okay, we've seen how to use the standalone.

IDE

enter distrobox container
in vscode on host, ctrl+shift+P, attach to running container

I changed docker path to `podman` for this, so i may need to go back and change that in the future.

Now setting up maven:

```
📦[b@junit-maven Test_Example]$ mvn archetype:generate -
DgroupId=com.mycompany.app -DartifactID=test-app -
DarchetypeArtifactID=maven-archetype-quickstart -DinteractiveMode=false
```

error:

```
[WARNING] Property artifactId is missing. Add -DartifactId=someValue
[INFO] ------------------------------------------------------------------------
---
[INFO] BUILD FAILURE
[INFO] ------------------------------------------------------------------------
---
[INFO] Total time:  20.458 s
[INFO] Finished at: 2026-04-27T11:18:28-04:00
[INFO] ------------------------------------------------------------------------
---
[ERROR] Failed to execute goal org.apache.maven.plugins:maven-archetype-
plugin:3.4.1:generate (default-cli)
```

running;

```
📦[b@junit-maven Test_Example]$ mvn archetype:generate -
DgroupId=com.mycompany.app -DartifactID=test-app -
DarchetypeArtifactID=maven-archetype-quickstart -DinteractiveMode=false -
DartifactId=Calculator
```

```
[INFO] ------------------< org.apache.maven:standalone-pom >----------------
---
[INFO] Building Maven Stub Project (No POM) 1
[INFO] --------------------------------[ pom ]---------------------------------
---
[INFO]
[INFO] >>> archetype:3.4.1:generate (default-cli) > generate-sources @
```

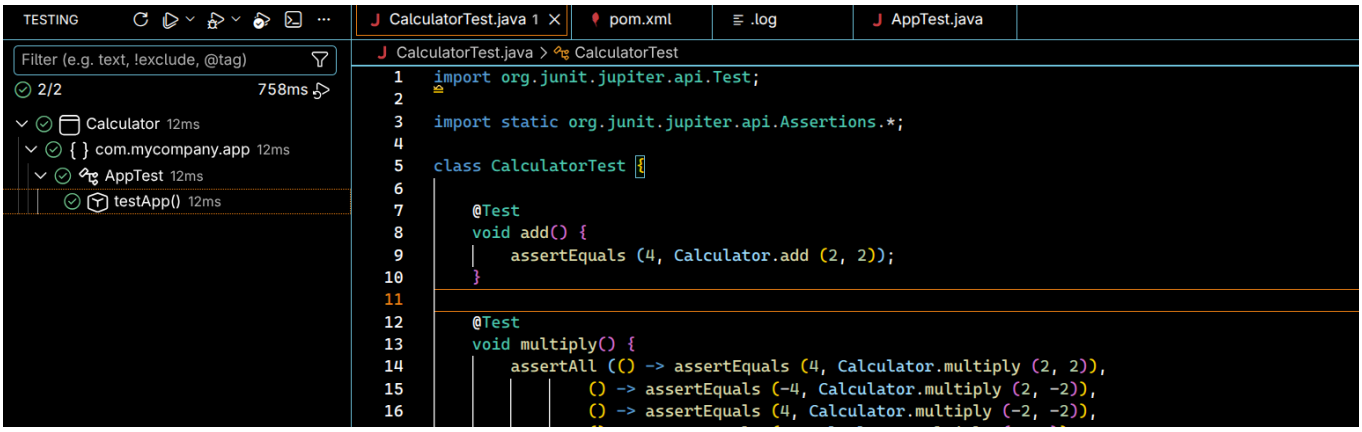
```
standalone-pom >>>
[INFO]
[INFO] <<< archetype:3.4.1:generate (default-cli) < generate-sources @
standalone-pom <<<
[INFO]
[INFO]
[INFO] --- archetype:3.4.1:generate (default-cli) @ standalone-pom ---
[INFO] Generating project in Batch mode
[INFO] No archetype defined. Using maven-archetype-quickstart
(org.apache.maven.archetypes:maven-archetype-quickstart:1.0)
[INFO] ----------------------------------------------------------------------
-------
[INFO] Using following parameters for creating project from Old (1.x)
Archetype: maven-archetype-quickstart:1.0
[INFO] ----------------------------------------------------------------------
-------
[INFO] Parameter: basedir, Value: /var/home/b/Documents/GitHub/Principles-
of-Programming-Concepts/Junit_notes/Test_Example
[INFO] Parameter: package, Value: com.mycompany.app
[INFO] Parameter: groupId, Value: com.mycompany.app
[INFO] Parameter: artifactId, Value: Calculator
[INFO] Parameter: packageName, Value: com.mycompany.app
[INFO] Parameter: version, Value: 1.0-SNAPSHOT
[INFO] project created from Old (1.x) Archetype in dir:
/var/home/b/Documents/GitHub/Principles-of-Programming-
Concepts/Junit_notes/Test_Example/Calculator
[INFO] ----------------------------------------------------------------------
---
[INFO] BUILD SUCCESS
[INFO] ----------------------------------------------------------------------
---
[INFO] Total time:  0.669 s
[INFO] Finished at: 2026-04-27T11:20:23-04:00
[INFO] ----------------------------------------------------------------------
---
```

Tests are passing.

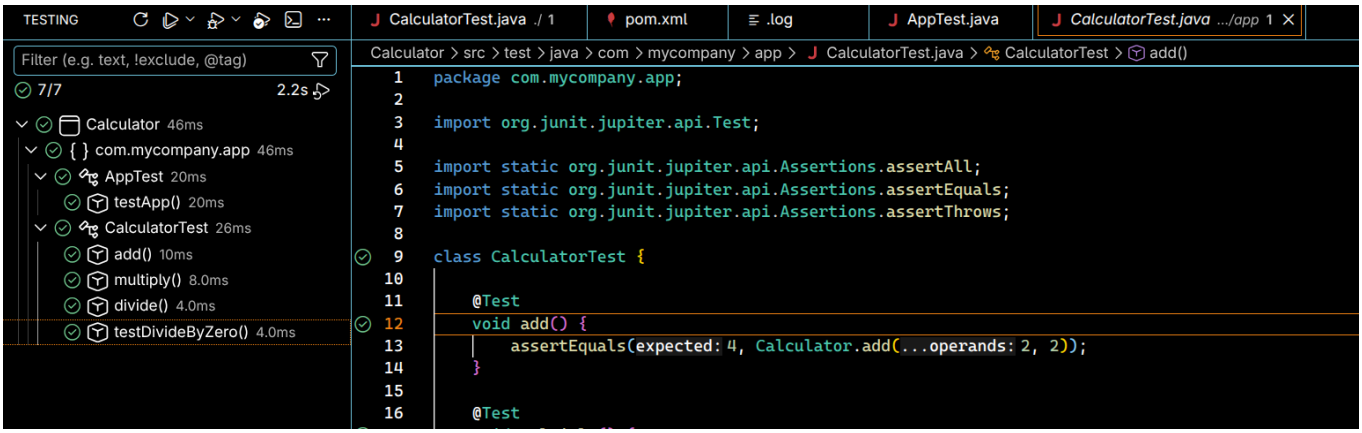
Copilot:

```
Switched this workspace to JUnit Jupiter already.

What I changed:

Removed legacy JUnit 3 dependency from pom.xml
Kept JUnit 5 dependency in pom.xml:
```

```
org.junit.jupiter:junit-jupiter:5.10.2 (test scope)
Converted old JUnit 3 style test class to JUnit 5 in AppTest.java:
package fixed to com.mycompany.app
imports now use org.junit.jupiter.api.Test
test method uses @Test with Jupiter assertions
```



So this is testing that true = true. I need to get the actual tests into the maven test source set. Right now they (CalculatorTest.java) is in the root directory. I've made copilot wire them into the maven test src module.



Now, we can see the tests included in CalculatorTest are running properly.